

Organizing Bears Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-organizing-bears-lab>  <https://github.com/learn-co-curriculum/python-p3-organizing-bears-lab/issues/new>

Learning Goals

- Use SQL to store data and retrieve it later on.
- Use SQLite to build relational databases on your computer.
- Perform CRUD operations on relational databases using SQL.

Key Vocab

- **SQL (Structured Query Language)**: a programming language that is used to manage relational databases and perform operations on the data that they contain.
- **Relational Database**: a collection of data that is organized in well-defined relationships. The most common type of database.
- **Query**: a statement used to return data from a database.
- **Table**: a collection of related data in a database. Composed of rows and columns. Similar to a class in Python.
- **Row**: a single record in a database table. Each column represents an attribute of the record. Similar to an object in Python.
- **Column**: a single field in a database table. Each row contains values in each column. Similar to a Python object's attributes.
- **Schema**: a blueprint of the construction of the tables in a database and how they relate to one another.

Lab Structure

This lab might seem a bit different than what you've seen before. Take a look at the file structure and read the comments to understand what each file is used for:

```
├── __init__.py      # designates "python-p3-organizing-bears-lab" as package
├── Pipfile
├── Pipfile.lock
├── README.md
├── lib
│   ├── __init__.py  # designates "lib" as package
│   ├── create.sql   # where you CREATE your schema
│   ├── insert.sql   # where you INSERT your data
│   ├── seed.sql     # data for in-memory test database
│   ├── sql_queries.py # where you write your SELECT queries
├── testing          # all the tests
│   ├── __init__.py  # designates "testing" as package
│   ├── create_test.py # this tests your create.sql file
│   ├── insert_test.py # this tests your insert.sql file
│   ├── select_test.py # this tests the queries you write in sql_queries.py
│   └── conftest.py   # configuration for pytest
```

This lab uses the `sqlite3` module from Python's standard library to allow us to connect to a SQL database from Python. How cool is that!? We'll use this module more in the lessons to come.

A Note on Testing

Let's briefly go over what is happening in setup blocks that our tests will be using.

```
connection = sqlite3.connect(":memory:")
```

```
cursor = connection.cursor()
```

```
create_file = open("lib/create.sql")
```

```
create_as_string = create_file.read()
cursor.executescript(create_as_string)
```

Before each test, two important things happen.

First, a new *in-memory* database is created. Why do we do this instead of creating a database file? Let's say we run our tests and they add ten items to our database. If we did not use an in-memory store, those would be in there forever. This way, our database gets thrown out after every running of the tests. You can learn more about in-memory databases [here](https://www.sqlite.org/inmemorydb.html)  (<https://www.sqlite.org/inmemorydb.html>).

Next, the test opens the `.sql` file, and runs the SQL code in that file in that in-memory database.

Part 1: CREATE TABLE

Get the tests in `testing/create_test.py` to pass by writing code in the `lib/create.sql` file. Your `CREATE` statement should look something like this:

```
CREATE TABLE bears (  
    //columns here  
);
```

Your columns should be the following types:

column	type
id	integer
name	text
age	integer
sex	text
color	text
temperament	text

column	type
alive	boolean

Read about [SQLite3 Datatypes](https://www.sqlite.org/datatype3.html)  [_ \(https://www.sqlite.org/datatype3.html\)](https://www.sqlite.org/datatype3.html) to determine what your insert values are going to be. Be sure to pay attention to how booleans are expressed in SQLite3.

Part 2: INSERT

Get the tests in `testing/insert_test.py` to pass by writing code in the `lib/insert.sql` file. Input the following 8 bears (you can make up details about them, but make sex either 'M' or 'F'):

- Mr. Chocolate
 - Rowdy
 - Tabitha
 - Sergeant Brown
 - Melissa
 - Grinch
 - Wendy
 - unnamed (refer back to how to create a record that doesn't have one value)
-

Part 3: SELECT

Get the tests in `testing/select_test.py` to pass. Note that for this section, the database will be seeded with external data from the `lib/seed.sql` file so don't expect it to reflect the data you added above.

Note: Since it's a Python file, write your queries as strings in the `global` scope in the `lib/sql_queries.py` file. For example, to pass the first test, your Python string should look like this:

```
select_all_female_bears_return_name_and_age = """
SELECT
    bears.name,
    bears.age
FROM bears
WHERE sex='F';
"""
```

You may be expected to use SQL statements that you're not particularly familiar with. Make sure you use the resources and Google to find the right statements.

Resources

- [SQL Tutorial - W3Schools](https://www.w3schools.com/sql/) 
- [Documentation - SQLite](https://www.sqlite.org/docs.html) 
- [SQLite - VisualStudio Marketplace](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite) 
- [SQLite Keywords - SQLite](https://www.sqlite.org/lang_keywords.html) 
- [SQL Datatypes](https://www.sqlite.org/datatype3.html) 
- [SQL GROUP BY](https://www.sqlite.org/lang_select.html#resultset) 
- [ZetCode sqlite3 Tutorial](http://zetcode.com/db/sqlite/) 

This tool needs to be loaded in a new browser window

Load Organizing Bears Lab (CodeGrade) in a new window